

PA1 - 3D Shapes in WebGL

Tair Asset, AIB-2401. Student ID: 242458. Assigned solid: square frustum. Back points shift 0.15 left and 0.15 up.

E1 - Depth test and draw order

Prediction. I expected the nearer faces to stay visible with depth testing on. With it off, faces drawn later should cover earlier ones. I did not expect S to change the normal view because the cube and frustum do not overlap. It should make a difference if I move the frustum over the cube.

Result. In the normal scene (Figures 1-3), turning depth OFF exposes the cube's blue back in the centre, green bottom, teal right, purple left, yellow top, and a thin orange part of the front. Pressing S changes the label, but I cannot see a difference in the shapes. I then moved the frustum 0.70 left for Figures 4-5. In the overlapping area, whichever solid I draw second covers the first.

Explanation. The cube faces are stored front, back, top, left, right, bottom. Without the depth test, faces later in that list cover earlier ones. With LEQUAL, the nearer surface wins. The two solids normally do not draw over the same pixels, so swapping their calls has no visible effect.

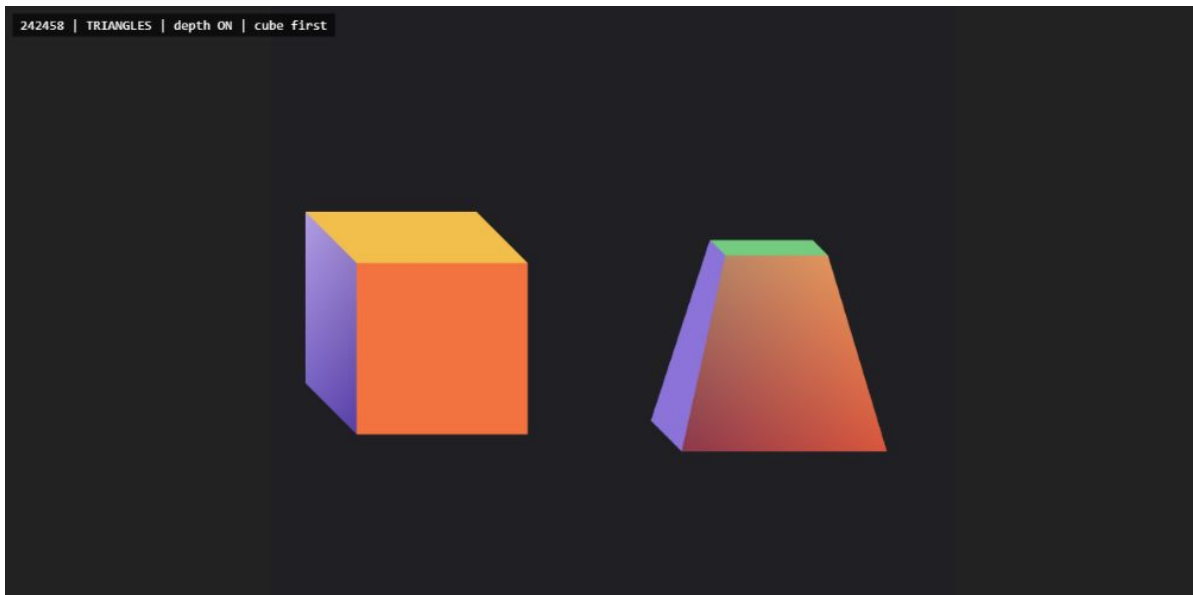


Fig 1. Normal scene, depth ON.

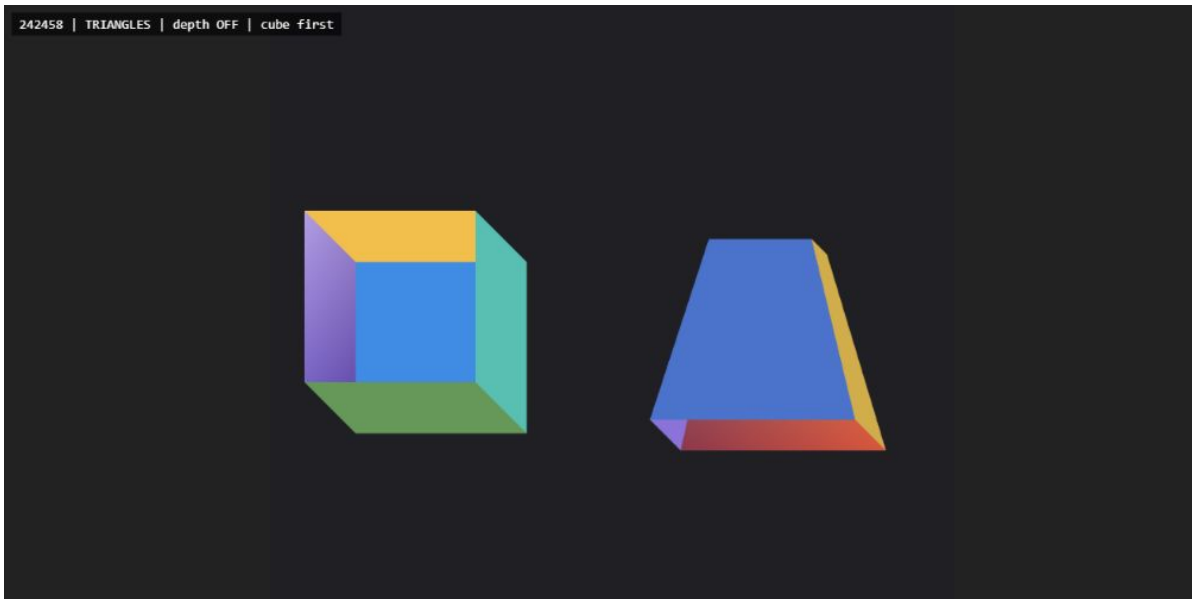


Fig 2. Normal scene, depth OFF, cube first.

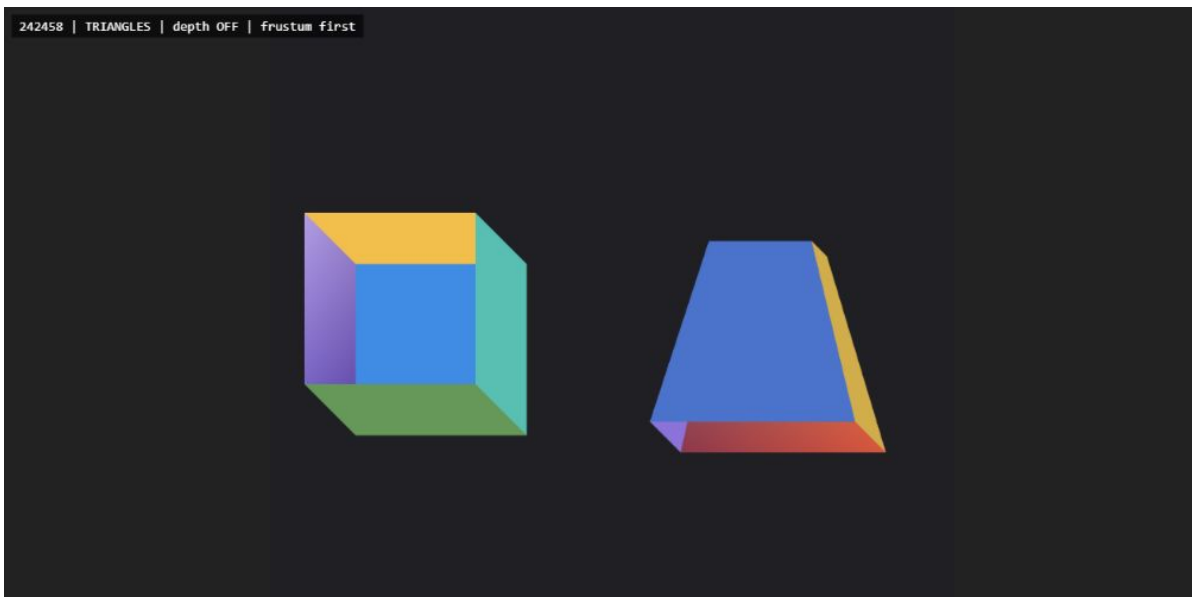


Fig 3. Normal scene, depth OFF, frustum first.

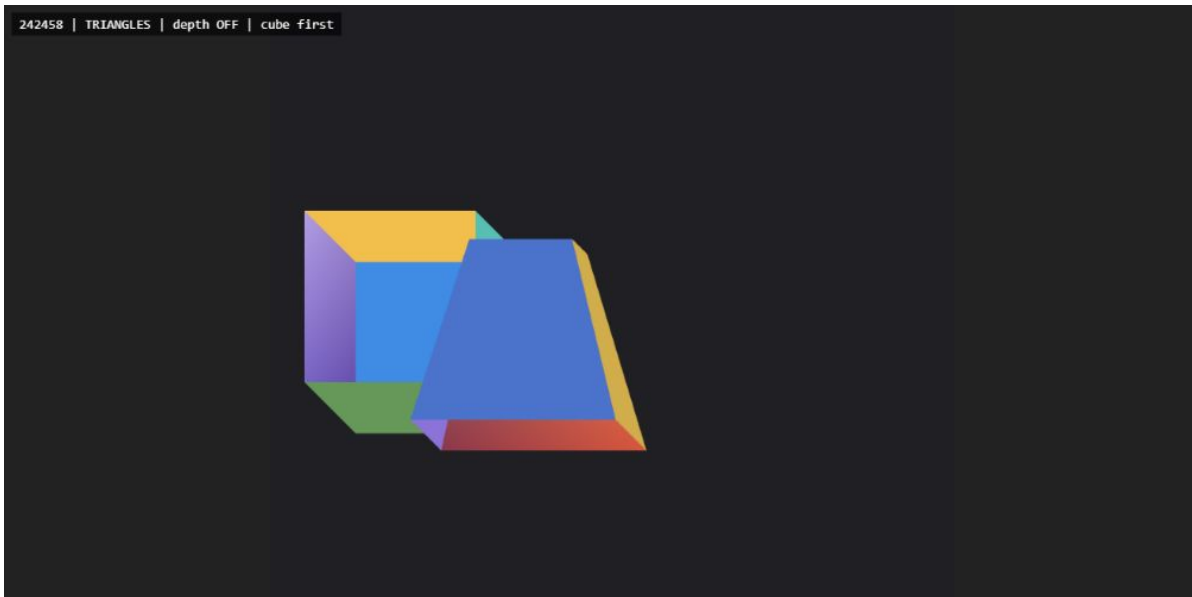


Fig 4. Temporary overlap, depth OFF, cube first.

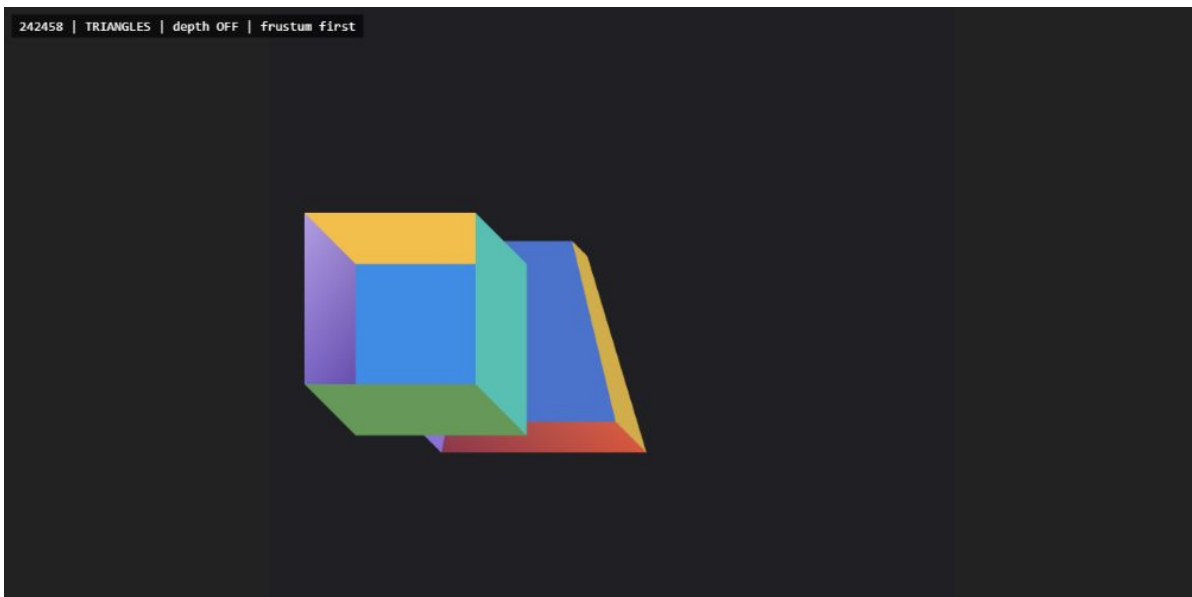


Fig 5. Temporary overlap, depth OFF, frustum first.

E2 - Why negative z is in front

Prediction. Our vertex shader passes the coordinates straight to clip space. Here $z = -0.5$ becomes depth 0.25, while $z = +0.5$ becomes 0.75. So I expected LEQUAL to keep the negative- z face in front. I expected GREATER alone to leave the canvas empty because the depth buffer starts at 1.0. With `clearDepth(0.0)` added, the farther faces should appear.

Result. GREATER alone gave me an empty scene, apart from the status label (Figure 6). After I set `clearDepth(0.0)`, the blue back faces and other farther surfaces appeared (Figure 7).

Explanation. The book describes z as increasing toward the viewer, but this program sends coordinates straight to clip space. WebGL maps clip-space z from $[-1, 1]$ to depth $[0, 1]$. With GREATER and an initial depth of 1.0, nothing can pass. Starting at 0.0 lets pixels pass, but larger depths then win, so back faces become visible.

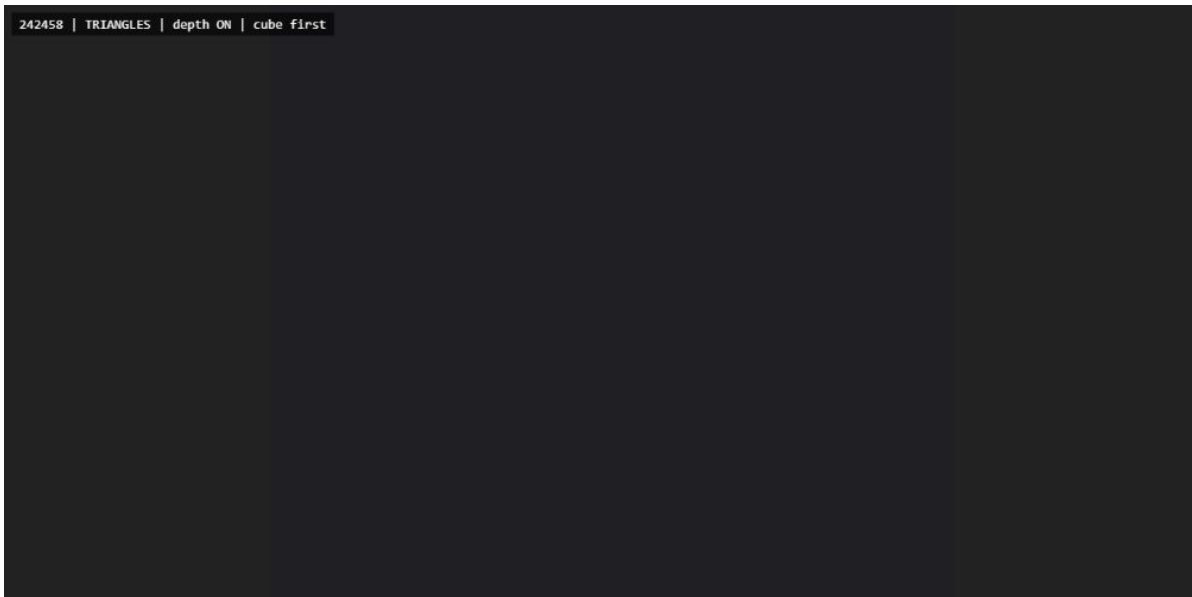


Fig 6. GREATER with the default clear depth 1.0.

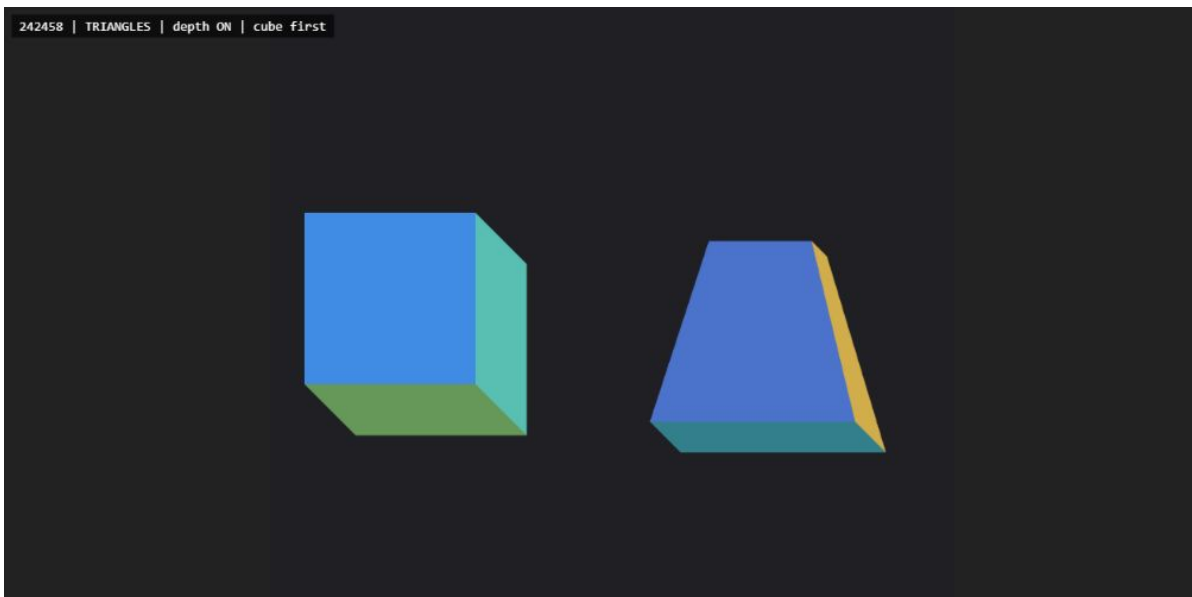


Fig 7. GREATER with `clearDepth(0.0)`.

E3 - Modes applied to triangle data

Prediction. Modes 2, 3 and 6 should look wrong because the vertex list was made for separate triangles.

Result. I got extra diagonals with `LINE_LOOP` (Figure 8), only some of the edges with `LINES` (Figure 9), and filled triangles in the wrong places with `TRIANGLE_STRIP` (Figure 10).

Explanation. `LINE_LOOP` connects consecutive vertices even when they belong to different faces. It also closes the loop with one last line. `LINES` takes vertices in pairs, so many triangle edges are skipped. `TRIANGLE_STRIP` keeps reusing the previous two vertices, which joins triangles that were never meant to touch.

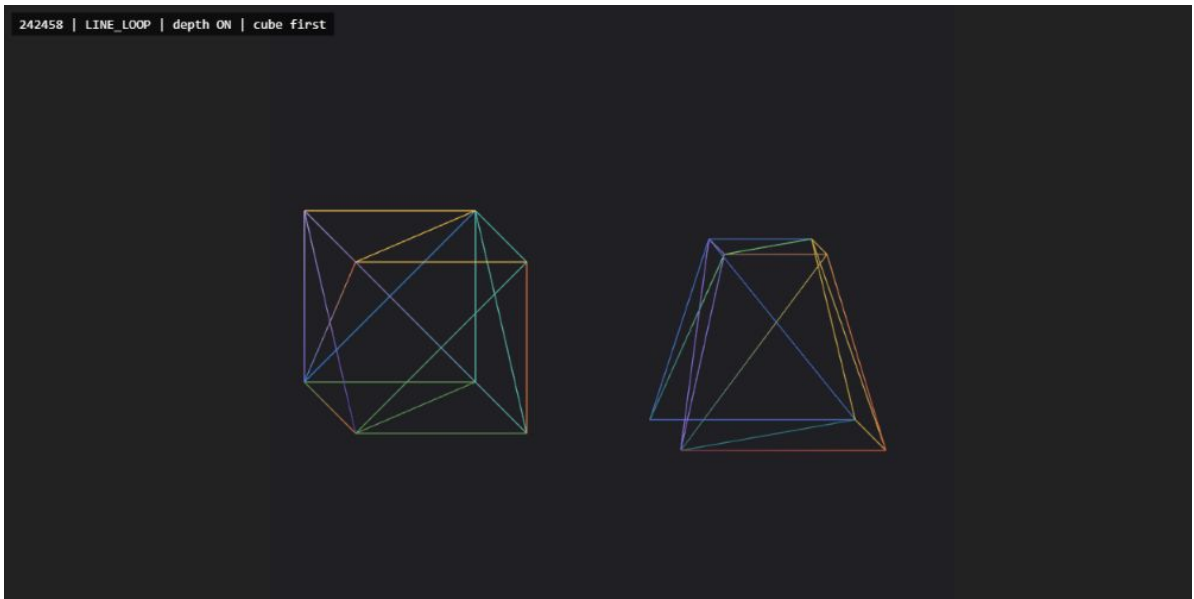


Fig 8. *LINE_LOOP*.

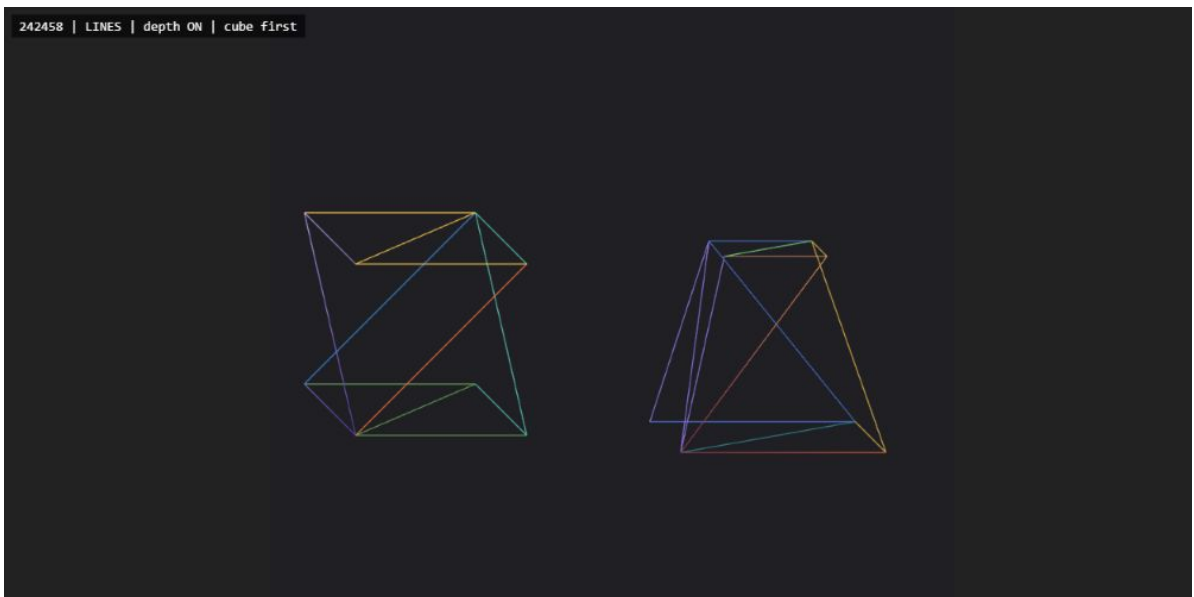


Fig 9. *LINES*.

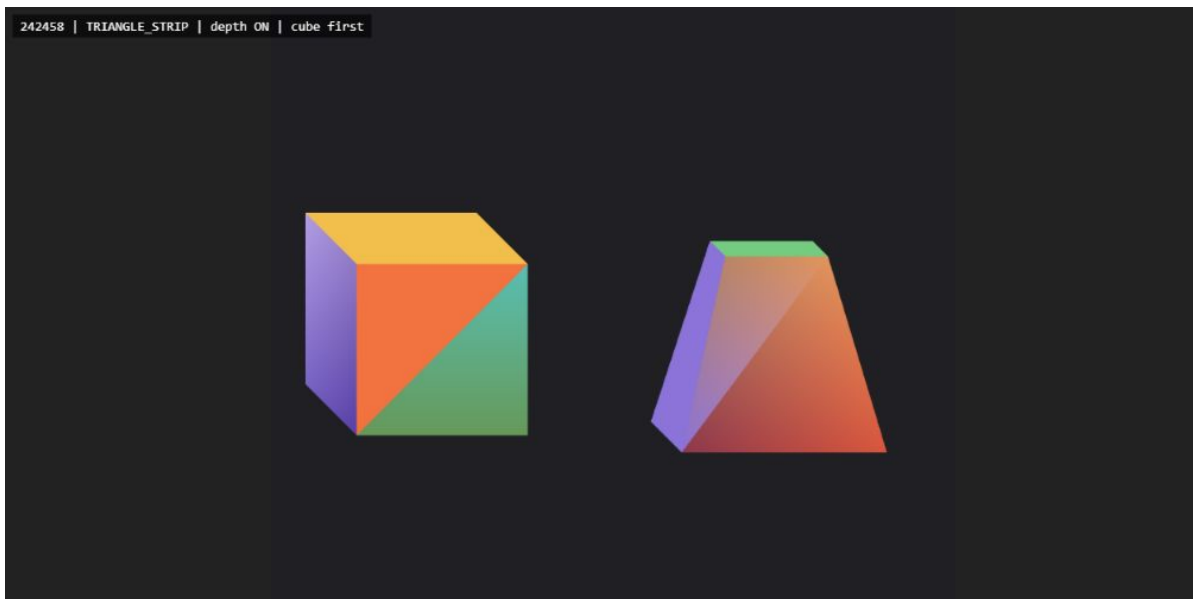


Fig 10. TRIANGLE_STRIP.

E4 - The missing buffer bind

Prediction. `initBuffers` leaves the colour buffer selected. If the position buffer is not bound before `vertexAttribPointer`, WebGL should read colours as positions. The drawing should break, but there should be no WebGL error.

Result. Instead of two solids, I got overlapping triangles near the upper-right area (Figure 11). The console still reported E4 pointer `gl.getError(): 0`.

Explanation. `vertexAttribPointer` uses the buffer selected at that moment. The colour values are still valid numbers, so WebGL can treat groups of three as x, y and z. The numbers are simply the wrong positions; that is why the picture is broken but the error code is zero.

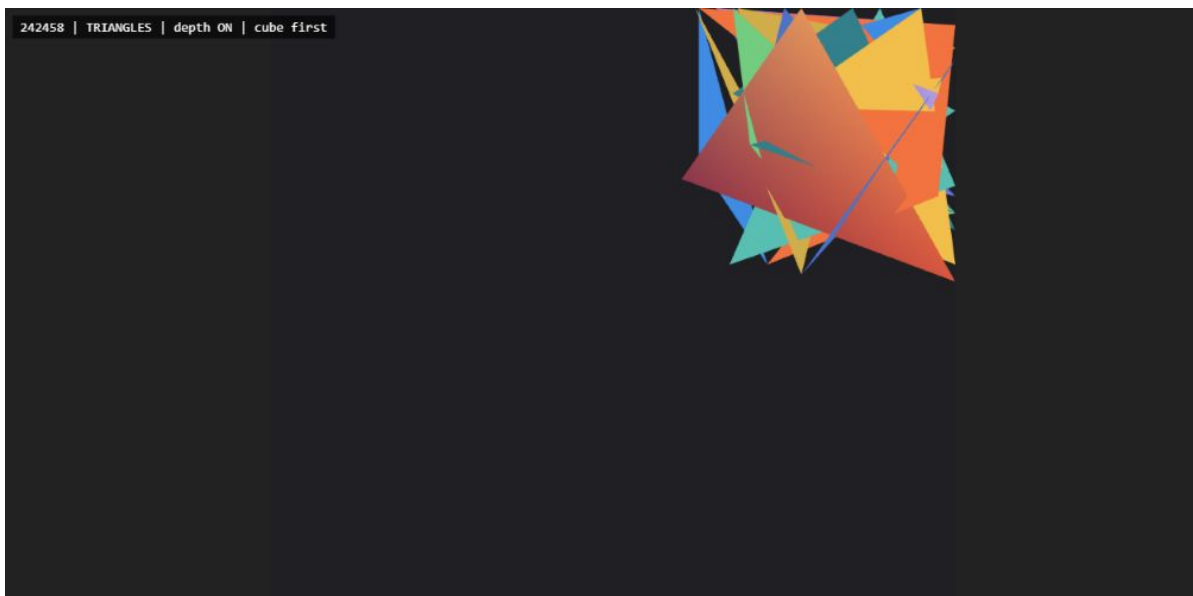


Fig 11. Position attribute reading the colour buffer.

E5 - Clipping

Prediction. Changing one cube vertex to $z = 1.2$ should cut its triangle at the far boundary. Changing that same entry to $x = 1.4$ instead should cut it at the right boundary.

Result. At $z = 1.2$, part of the orange front triangle disappeared and I could see the surfaces behind it (Figure 12). At $x = 1.4$, the triangle stretched toward the right edge, but nothing appeared outside the WebGL area (Figure 13).

Explanation. In this scene the clip-space limits are -1 and +1 on each axis. WebGL cuts the triangle at the limit instead of drawing the part outside. The same cube corner appears more than once in the array, but only one copy was changed for this test.

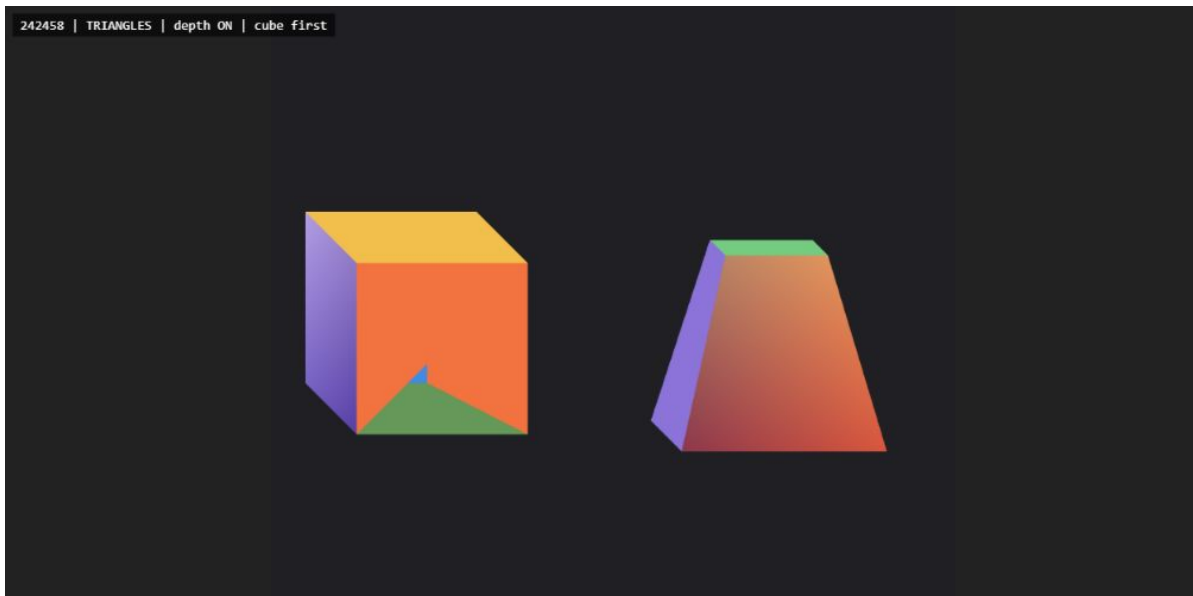


Fig 12. One cube vertex changed to $z = 1.2$.

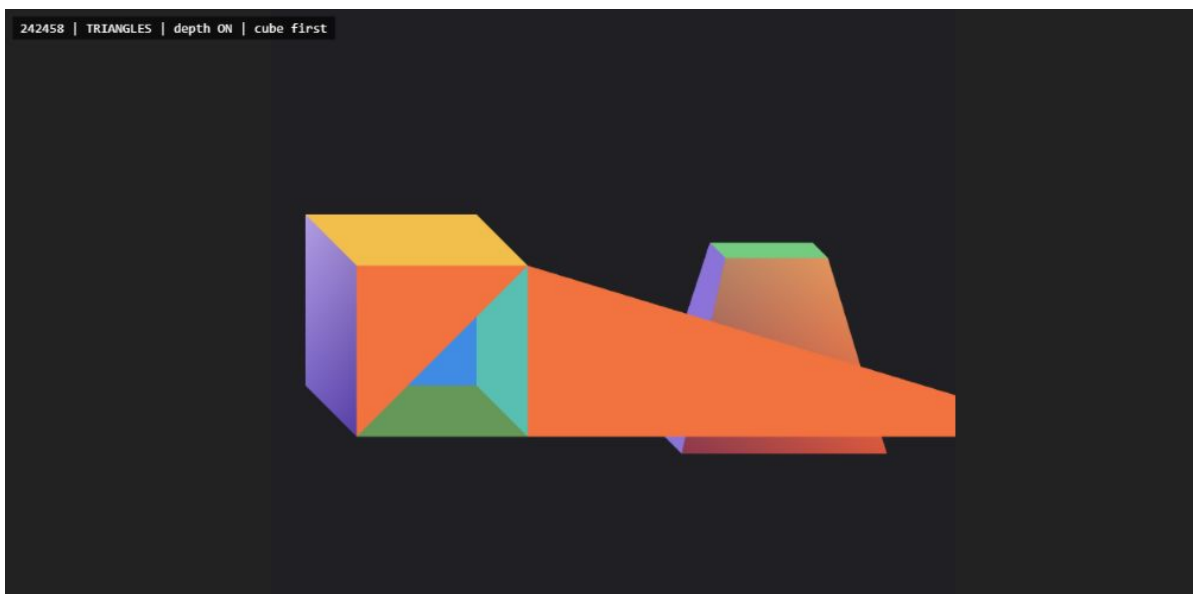


Fig 13. One cube vertex changed to $x = 1.4$.

E6 - Buffer sizes

Prediction. The cube and frustum use 36 vertices each, so $N = 72$. Positions should use $72 \times 3 \times 4 = 864$ bytes, and RGBA colours should use $72 \times 4 \times 4 = 1,152$ bytes. Each Float32 number takes four bytes.

Result. I selected each buffer and checked its size with `gl.getBufferParameter(gl.ARRAY_BUFFER, gl.BUFFER_SIZE)`. The console gave:

```
E6 total vertices: 72  
E6 position bytes: 864  
E6 color bytes: 1152
```

Explanation. The numbers match the calculation: three position values and four colour values per vertex, all stored as 32-bit floats.

Development log

When I first opened the starter page, the browser tab still said [Student ID].

I noticed it while checking whether the page loaded.

I replaced the placeholder with 242458 in `index.html` in commit 742f19f.